

AWS Academy

Cloud

Architecting 3 –

Capstone Project

Overview and objectives

Throughout this course, you have completed hands-on labs. You have used different AWS services and features to create compute instances, install operating systems (OSs) and software, deploy code, and secure resources. You practiced how to enable load balancing and automatic scaling and how to architect for high availability to build lab-specific applications.

In this project, you're challenged to use familiar AWS services to build a solution without step-by-step guidance. Specific sections of the assignment are meant to challenge you on skills that you have acquired throughout the Academy Cloud Architecting (ACA) course.

By the end of this project, you should be able to apply the architectural design principles that you learned in this course to do the following:

- Create a database (DB) instance that the PHP application can query.
- Create and deploy the highly available PHP application with the load distributed across multiple web servers and Availability Zones.
- Use AWS Secrets Manager.
- Import data into a MySQL database from a SQL dump file.
- Secure the application to prevent public access to application servers and backend systems.

The lab environment and monitoring your budget

This environment is long lived. When the session timer gets to 0:00, the session will end, but any data and resources that you created in the Amazon Web Services (AWS) account will be retained. If you later launch a new session (for example, the next day), you will find that your work is still in the lab environment. Also, at any point before the session timer reaches 0:00, you can choose **Start Lab** again to extend the lab session time.

Important: Monitor your lab budget in the lab interface. When you have an active lab session, the latest known remaining budget information displays at the top of this screen. This data comes from AWS Budgets, which typically updates every 8–12 hours. Therefore, the remaining budget that you see might not reflect your most recent account activity. If you exceed your lab budget, your lab account will be disabled, and all progress and resources will be lost. Therefore, it's important for you to manage your spending.

The following are a few suggestions to avoid overspending:

- Launch only the number of EC2 instances that you need, sized to your requirements.
- Delete resources and data that you no longer need.
- Typically, it is compute-type resources, such as Amazon Elastic Compute Cloud (Amazon EC2) instances, Amazon Relational Database Service (Amazon RDS) database instances, AWS Cloud9, and NAT gateway instances, that you leave running that most quickly use up your budget. Turn these off when you no longer need them, or—better yet—delete them.
- Stop any NAT gateways, EC2 instances, and Amazon RDS instances when they are not in use.
 - Be aware that if you do stop an Amazon RDS instance or cluster and leave it stopped for 7 days, AWS might start it again automatically, which will increase the cost impact.
 - NAT gateway resources, if left running, will also use up your budget.
- To estimate your costs, use the [AWS Pricing Calculator](#).

Here is a sample that shows the cost to run the resources in the us-east-1 Region for a month and an entire year:



Note: Pricing is subject to change. The calculation in the previous sample is an example from a point in time.

If you want to reset your AWS account to the original settings before you ran sessions of this lab environment, choose **Reset**. **CAUTION:** If you choose **Reset** and then choose **Yes** to confirm that you want to reset, you will permanently delete everything that you have created or stored in the AWS account.

AWS service restrictions

In this lab environment, access to AWS services and service actions might be restricted to the ones that are needed to complete the lab instructions. You might encounter errors if you attempt to access other services or perform actions beyond the ones that are described in this lab.

Scenario

Example Social Research Organization is a (fictitious) nonprofit organization that provides a website for social science researchers to obtain global development statistics. For example, visitors to the site can look up various data points, such as the life expectancy for any country in the world over the past 10 years.

Shirley Rodriguez, a researcher at the nonprofit organization, developed the website. She thought it would be valuable to share the data that she had gathered with other researchers. Shirley stores the data in a MySQL database, and the data is available through a PHP website that she built. She initially published the site through a commercial hosting company that provides limited support for technical issues and security.

Over the past year, Shirley's website has grown in popularity. As a result of increased traffic, she started receiving complaints that the site is not as responsive as it used to be. She also experienced an attempted ransomware security breach. The security breach was unsuccessful, but her supervisor, Mateo Jackson, suggested that Shirley investigate new ways to host the website.

Shirley heard about AWS and initially moved her website and database to an EC2 instance that runs in a public subnet. She also runs an instance of MySQL on the same EC2 instance.

Shirley approached your team to make sure that her current design follows architectural best practices. She wants to make sure that she has a robust and secure website. One of your colleagues started the process of migrating the site to a more secure implementation, but they were reassigned to another project. Your tasks are to complete the implementation, make sure that the website is secure, and confirm that the website returns data from the query page.

The following summary lists the solution requirements and provides a diagram of the current environment.

Solution requirements

- Provide secure hosting of the MySQL database.
- Provide secure access to the database.
- Provide anonymous access to web users.
- Run the website on a t2.micro EC2 instance running in private subnets and provide Secure Shell (SSH) access to administrators.
- Provide high availability to the website through a load balancer.
- Provide automatic scaling that uses a launch template.

As part of the lab provisioning, the components in the following architecture diagram are already available to you in the lab:

Starting architecture



Assumptions

This project will be built in a controlled lab environment that has restrictions on services, features, and budget. Consider the following assumptions for the project:

- The application is deployed in one AWS Region (the solution does not need to be multi-Regional).
- The website does not need to be available over HTTPS or a custom domain.
- The solution is deployed on Amazon Linux 2023 machines by using the PHP code that is provided. A template is already available in the lab to launch EC2 instances.

- The solution uses the PHP code as provided unless the instructions specifically direct you to change the code.
- The solution uses services and features within the restrictions of the lab environment.
- The solution uses all the three pairs of subnets provisioned.
- The solution uses the available AWS Identity and Access Management (IAM) role.
- The database can be hosted in a single Availability Zone or multiple Availability Zones.
- Store database connection information in Secrets Manager.
- The website is publicly accessible without authentication.

Disclaimer: A security best practice is to allow access to the website through authentication. However, because you are building this application as a part of the course, those features are beyond the scope of this project.

Project deliverables

To complete this assignment, you must do the following:

- Submit a diagram that illustrates your solution.
- Deploy and demonstrate a PHP application that meets the system requirements outlined in these lab instructions.
- Submit a written summary of the design decisions that you made to achieve the result.

Approach

Recommendation: Develop your project solution that is mentioned in the following steps. This will help you map the building blocks with the solution requirements.

Access the AWS Management Console

1. At the top of these instructions, choose **Start Lab** to start the lab session.
 - The lab session starts.
 - A timer displays at the top of the page and shows the time remaining in the session.
2. **Tip:** To refresh the session length at any time, choose **Start Lab** again before the timer reaches 0:00.
 - Before you continue, wait until the circle icon to the right of the [AWS](#) link in the upper-left corner turns green.

3. To connect to the AWS Management Console, choose the **AWS** link in the upper-left corner above the terminal window.
 - A new browser tab opens and connects you to the console.
Tip: If a new browser tab does not open, a banner or icon is usually at the top of your browser with a message that your browser is preventing the site from opening pop-up windows. Choose the banner or icon, and then choose **Allow pop-ups**.

Step 1: Creating an Amazon RDS MySQL database

3. Create an Amazon RDS MySQL database.

Tips:

- The database security group should allow only the web application to access the database.
- Use Secrets Manager to store the username and password.
- Keep the initial database as **countries**.
- Use the database subnet group already provisioned in the lab.

Note: Turning on additional features while configuring the database might incur charges that impact your cost estimate. Turn on only the features that are essential to your work.

References:

- AWS Academy Cloud Architecting – Lab: Guided Lab – Creating an Amazon RDS Database
- [Create and Connect to a MySQL Database with Amazon RDS](#)

Step 2: Creating an Application Load Balancer

4. Launch an Application Load Balancer in the available public subnets. The load balancer's DNS name will be used to access your web application.

Tip: Use a minimum of two Availability Zones.

References:

- [Getting Started with Elastic Load Balancing](#)

- AWS Academy Cloud Architecting – Lab: Creating a Highly Available Environment

Step 3: Creating the application server in an Auto Scaling Group

5. Create a launch template that you will use to launch EC2 instances in an Auto Scaling group.
6. Use an Auto Scaling group to launch the EC2 instances (application servers) that host the web application.
7. To install the required web application on the virtual machine, use the [PHP code that is available](#).
8. Use the Application Load Balancer that you created earlier.

Note: An EC2 launch template named **Example-LT** is provided in the AWS account. It already has both the SQL dump file and the unzipped PHP application resources built into it.

Tips:

- Use a target tracking policy.
- Set the Auto Scaling group size according to your estimated requirements.
- You can use the default values (for example, for group size and CPU utilization) initially and then adjust them later as needed.

Reference:

- AWS Academy Cloud Architecting – Lab: Creating a Highly Available Environment

Step 4: Importing data into an RDS MySQL database

9. Migrate the data from the original database, which is on an EC2 instance, to the new Amazon RDS database. Use the [data dump file that is available](#).
10. Refer to Secrets Manager for the username and password.
11. Use the same database that you created while provisioning an RDS MySQL database.

Reference:

- For commands, see "[Importing Data to an Amazon RDS MariaDB or MySQL Database with Reduced Downtime](#)."

Step 5: Testing the application and reviewing imported data

12. To test the application, access the web application from a browser by using the load balancer URL.
13. To review and query the imported data, use the query functionality in the application.

Submitting your work

14. To record your progress, at the top of these instructions, choose **Submit**.
15. When prompted, choose **Yes**.

After a couple of minutes, the **Grades** panel appears and shows you how many points you earned for each task. If the results don't display after a couple of minutes, choose **Grades** at the top of these instructions.

Tip: You can submit your work multiple times. After you change your work, choose **Submit** again. Your last submission is recorded for this lab.

Ending your session

Reminder: This is a long-lived lab environment. Data is retained until either you use the allocated budget or the course end date is reached (whichever occurs first).

To preserve your budget when you are finished for the day or when you are finished actively working on the assignment for the time being, do the following:

16. At the top of this page, choose **End Lab**, and then choose Yes to confirm that you want to end the lab.

A message appears: *Ended AWS Lab Successfully*

Note: Choosing **End lab** in this lab environment will not delete the resources that you have created. They will still be there the next time you choose **Start Lab** (for example, on another day).

© 2023 Amazon Web Services, Inc. and its affiliates. All rights reserved. This work may not be reproduced or redistributed, in whole or in part, without prior written permission from Amazon Web Services, Inc. Commercial copying, lending, or selling is prohibited.